

GenAI Document Assistant — Capstone Documentation

Course: Edureka PGP in Generative AI & ML (Batch B9)

Project: Capstone Project ILT — AI Agent-Based Knowledge & Decision Support System

Submission Date: July 2026

GitHub: <https://github.com/coolksrini/genai-doc-assistant>

Table of Contents

1. Project Overview
 2. System Architecture
 3. RAG Pipeline Walkthrough
 4. Agent Roles & Workflow
 5. System Setup
 6. API Documentation
 7. Deployment Guide
 8. Testing Approach
 9. Limitations
 10. Challenges & How They Were Solved
 11. Assumptions
 12. Future Improvements
-

1. Project Overview

What It Does

The GenAI Document Assistant is an AI-powered enterprise document Q&A system. Users upload documents in any format — PDF, TXT, CSV, Excel, JSON, or YAML — and ask natural language questions. The system retrieves the most relevant content and generates answers grounded **exclusively** in the uploaded documents. If an answer cannot be found, the system explicitly says so rather than hallucinating.

Why It Matters

Enterprise knowledge is locked in documents — reports, spreadsheets, policies, technical specs. Traditional search returns documents; this system returns **answers**. By combining RAG (Retrieval-Augmented Generation) with autonomous AI agents, it:

- Eliminates manual document trawling
- Prevents hallucination through grounding enforcement
- Maintains a full audit trail (agent trace) for every answer
- Works offline with local models — no cloud API key required

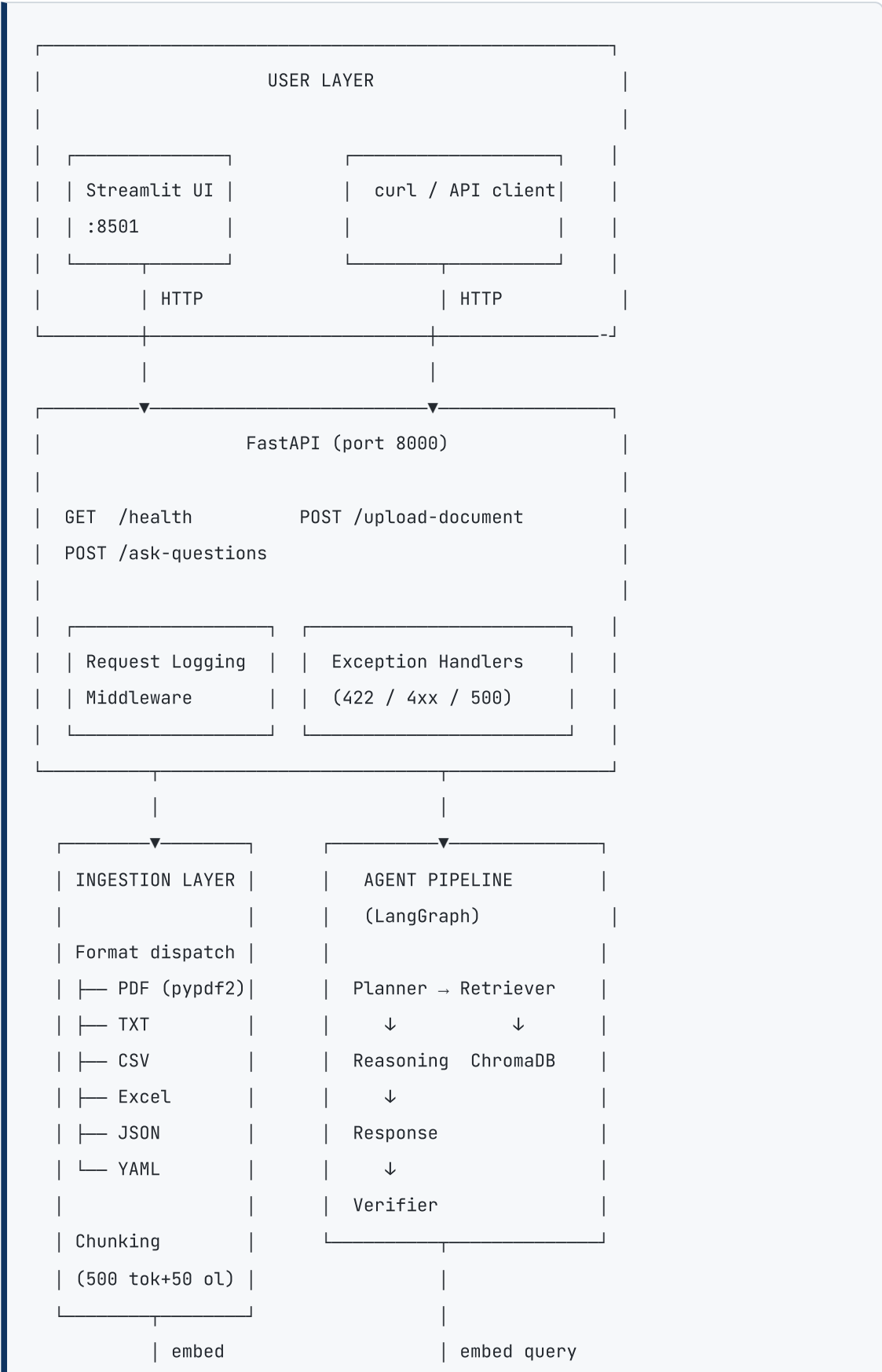
Design Philosophy

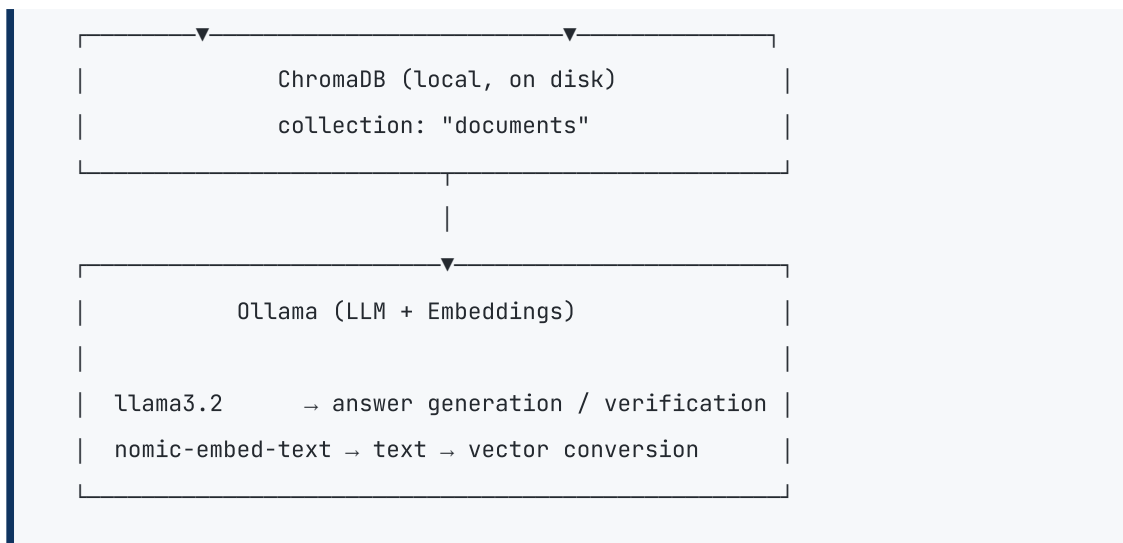
The system was built with three guiding principles:

1. **Spec-first development:** formal requirements and user stories defined before any code was written, ensuring every feature traces back to a documented need.
 2. **Separation of concerns:** document parsing, chunking, embedding, and agent reasoning are isolated into distinct modules — each independently testable and replaceable without affecting the rest.
 3. **Local-first, cloud-ready:** all inference runs locally via Ollama (no API keys, no cost, full privacy). Switching to a cloud provider requires only two environment variable changes.
-

2. System Architecture

High-Level Architecture





Component Responsibilities

Component	File(s)	Responsibility
FastAPI App	<code>main.py</code>	HTTP server, middleware, exception handling
Routes	<code>app/api/routes.py</code>	Request validation, orchestration, response models
Config	<code>app/core/config.py</code>	Env var loading via pydantic Settings
LLM Factory	<code>app/core/llm.py</code>	<code>get_llm()</code> + <code>get_embeddings()</code> — provider-agnostic
Ingestion	<code>app/services/ingestion.py</code>	Format-specific document loaders
Chunking	<code>app/services/chunking.py</code>	Text splitting with overlap
Embedding	<code>app/services/embedding.py</code>	ChromaDB wrapper, batched upserts
Agent Graph	<code>app/agents/graph.py</code>	LangGraph StateGraph definition
Planner	<code>app/agents/planner.py</code>	Input validation, injection detection
Retriever	<code>app/agents/retriever.py</code>	Cosine similarity search
Reasoning	<code>app/agents/reasoning.py</code>	Context synthesis from chunks
Response	<code>app/agents/response.py</code>	Grounded LLM call + verifier
Logger	<code>app/utils/logging.py</code>	Structured JSON logging
Streamlit UI	<code>ui/streamlit_app.py</code>	Web frontend, HTTP API client

3. RAG Pipeline Walkthrough

Document Ingestion Flow

```
User uploads annual_report.pdf (2MB)
|
▼
API validates: format=.pdf ✓, size=2MB < 10MB ✓
|
▼
ingestion.py: PdfReader extracts text page by page
| → [Document(page_content="Q3 revenue...", metadata={source: "annual
|
|
▼
chunking.py: RecursiveCharacterTextSplitter
| chunk_size=500 tokens, chunk_overlap=50
| → 47 chunks, each with chunk_index in metadata
|
|
▼
embedding.py: nomic-embed-text converts each chunk → 768-dim vector
| Batched: 100 chunks/call (avoids Ollama timeout on large docs)
|
|
▼
ChromaDB stores: {text, embedding, metadata: {source, chunk_index, page}}
|
|
▼
Response: {"document_id": "...", "chunk_count": 47, "status": "ingested"}
```

Query Flow

User asks: "What was the Q3 revenue?"

|

▼

API validates: question non-empty ✓, documents exist in store ✓

|

▼

LangGraph initialises AgentState:

```
{question, top_k=5, chunks=[], context="", answer="", sources=[], is_grounded=
```

|

▼

PLANNER

- Strips whitespace, checks 15 injection patterns
- Appends: "Planner: Analysing question – 'What was...'"
- Passes state to Retriever

|

▼

RETRIEVER

- Embeds question: nomic-embed-text → 768-dim vector
- ChromaDB cosine similarity → top-5 chunks
- Populates: state["chunks"], state["sources"]
- Appends: "Retriever: Retrieved 5 chunk(s)"

|

▼

REASONING

- Formats chunks into labelled context string:
"[Source 1: annual_report.pdf chunk 12]\nQ3 revenue..."
- Populates: state["context"]
- Appends: "Reasoning: Synthesised context from 5 chunk(s)"

|

▼

RESPONSE

- System prompt: "Answer ONLY from the context below..."
- LLM call: llama3.2 → "Q3 revenue was \$4.2M"
- Checks: is refusal message in answer? → No → is_grounded=True
- Appends: "Response: Generated answer (grounded=True)"

```
| (conditional: skip verifier if refusal)
▼
┌── VERIFIER ────────────────────────────────────────────────────────────────────────────────────┐
| • Second LLM call: "Is this answer supported? YES/NO" |
| • Verdict: "YES" → is_grounded confirmed              |
| • If "NO" → overrides answer to refusal, clears sources |
| • Appends: "Verifier: Answer confirmed grounded (YES)" |
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
|
▼
Response: {
  "answer": "Q3 revenue was $4.2 million.",
  "is_grounded": true,
  "sources": [{"document_name": "annual_report.pdf", "chunk_index": 12, "excerpt": "..."},
  "agent_trace": ["Planner: ...", "Retriever: ...", "Reasoning: ...", "Response: ..."]
}
```

4. Agent Roles & Workflow

The Five Agents

Planner

Purpose: Gate-keeper — validates every question before any expensive computation.

What it does: 1. Strips whitespace from the question 2. Checks against 15 prompt injection patterns (ignore/disregard/jailbreak/pretend/act-as/reveal/override...) 3. Raises `ValueError` (caught at API layer → HTTP 400) if unsafe 4. Appends a trace entry and passes state to Retriever

Why a separate agent: Separation of concerns — injection detection logic is independently testable and updatable without touching retrieval or generation.

Retriever

Purpose: Find the document chunks most relevant to the user's question.

What it does: 1. Embeds the question using the same model used at ingestion (nomic-embed-text) 2. Performs cosine similarity search in ChromaDB → top-K results (default K=5) 3. Populates `chunks` (for Reasoning) and `sources` (for the final response)

Key design choice: Same embedding model for both ingestion and query — this is critical for semantic alignment. If you change the embed model, you must re-ingest all documents.

Reasoning

Purpose: Transform raw retrieved chunks into a clean, structured context string.

What it does: 1. Labels each chunk with its source document and position: `[Source 1: report.pdf chunk 12]` 2. Joins chunks with separators 3. Stores the result as `state["context"]` for the Response agent

Why a separate agent: The LLM performs better with a well-structured context. This step can be enhanced independently — e.g., adding cross-chunk deduplication or relevance scoring — without touching the LLM call.

Response

Purpose: Generate the final answer using only the retrieved context.

What it does: 1. If context is empty: immediately returns the canonical refusal message 2. Otherwise: calls the LLM with a strict system prompt — "Answer ONLY from the context below. If the answer is not present, respond with exactly: [refusal message]" 3. Checks if the LLM's output contains the refusal string → sets `is_grounded` 4. Clears `sources` if not grounded (no citation for a refusal)

The grounding guarantee: The system prompt is the primary grounding mechanism. The LLM is explicitly forbidden from using training data. The Verifier provides a second check.

Verifier

Purpose: Independent second opinion on whether the answer is actually supported by the context.

What it does: 1. Skips immediately if `is_grounded=False` (already a refusal — no wasted LLM call) 2. Sends a separate LLM request: VERIFY_PROMPT → "Is this answer supported by the context? Reply YES or NO" 3. If verdict starts with "NO": overrides answer to refusal, sets `is_grounded=False`, clears sources 4. If "YES": confirms grounding, passes through

Why needed: Small local models (llama3.2 3B) occasionally produce answers that look grounded but aren't. The Verifier catches these cases with a focused binary question.

Agent State Schema

```
class AgentState(TypedDict):
    question: str          # user input (never modified after Planner)
    top_k: int             # number of chunks to retrieve (default 5)
    plan: list[str]        # reserved for future multi-step planning
    chunks: list[dict]      # Retriever output: [{text, source, chunk_index, page}]
    context: str           # Reasoning output: labelled context string
    answer: str            # Response/Verifier output: final answer text
    sources: list[dict]     # [{document_name, chunk_index, excerpt}]
    is_grounding: bool     # True only if answer comes from retrieved context
    agent_trace: list[str] # audit log: one entry per agent step
```

LangGraph Execution Flow



5. System Setup

Prerequisites

Requirement	Version	Notes
Python	3.11+	3.14 also tested
Ollama	Any	Must use desktop app, not brew formula
Git	Any	

Step 1: Install Ollama and Pull Models

Download Ollama desktop app from <https://ollama.com> (use the `.app` installer, not `brew install ollama` — the brew formula is missing required binaries).

```
ollama pull llama3.2          # 2.0 GB – chat model
ollama pull nomic-embed-text # 274 MB – embedding model
ollama list                   # verify both are listed
```

Step 2: Clone and Configure

```
git clone https://github.com/coolksrini/genai-doc-assistant
cd genai-doc-assistant

python -m venv .venv
source .venv/bin/activate      # macOS/Linux
# .venv\Scripts\activate      # Windows

pip install -r requirements.txt
cp .env.example .env          # defaults work for local Ollama
```

Step 3: Start the Application

```
# Terminal 1 – FastAPI backend
python main.py
# → http://localhost:8000
# → http://localhost:8000/docs (interactive API docs)

# Terminal 2 – Streamlit UI
streamlit run ui/streamlit_app.py
# → http://localhost:8501
```

Step 4: Upload a Document and Ask a Question

```
# Upload a document
curl -X POST http://localhost:8000/upload-document \
  -F "file=@data/sample_docs/attention_is_all_you_need.pdf"

# Ask a question
curl -X POST http://localhost:8000/ask-questions \
  -H "Content-Type: application/json" \
  -d '{"question": "What is the attention mechanism?"}'
```

Environment Variables

All configuration is via environment variables. Copy `.env.example` to `.env` and modify as needed:

```
# LLM – change these two to use OpenAI, Groq, or any OpenAI-compatible provider
LLM_BASE_URL=http://localhost:11434/v1    # Ollama default
LLM_API_KEY=ollama                        # placeholder for Ollama
LLM_MODEL=llama3.2

# Embeddings
EMBED_MODEL=nomic-embed-text

# Storage
CHROMA_PATH=./data/chroma_db

# Upload limits
MAX_FILE_SIZE_MB=10

# Chunking
CHUNK_SIZE=500
CHUNK_OVERLAP=50
```

Switching to a cloud LLM (no code changes required):

```
# Groq (fast, free tier)
LLM_BASE_URL=https://api.groq.com/openai/v1
LLM_API_KEY=gsk_your_key_here
LLM_MODEL=llama-3.3-70b-versatile
```

6. API Documentation

The FastAPI server exposes interactive documentation at

`http://localhost:8000/docs`.

GET /health

System liveness and readiness probe.

Always returns HTTP 200 — the `status` field signals degradation.

```
{
  "status": "ok",          // "ok" | "degraded"
  "version": "1.0.0",
  "llm": "available",     // "available" | "unavailable"
  "vector_store": "ready" // "ready" | "empty" | "unavailable"
}
```

`status` is `"degraded"` when LLM is unreachable or vector store is unavailable.

POST /upload-document

Upload and ingest a document.

Request: `multipart/form-data`, field name: `file`

Accepted formats: `.pdf`, `.txt`, `.csv`, `.xlsx`, `.json`, `.yaml`

Size limit: 10 MB

Response 200:

```
{
  "document_id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "annual_report.pdf",
  "chunk_count": 47,
  "status": "ingested"
}
```

Error responses: - `413` — file exceeds 10 MB limit - `415` — unsupported file format
- `500` — ingestion failed (corrupt file or Ollama unavailable)

POST /ask-questions

Ask a natural language question against all ingested documents.

Request:

```
{
  "question": "What was the Q3 revenue?",
  "top_k": 5
}
```

`top_k` (optional, default 5, range 1–20): number of document chunks to retrieve.

Response 200 — grounded answer:

```
{
  "question": "What was the Q3 revenue?",
  "answer": "According to the document, Q3 revenue was $4.2 million.",
  "is_grounding": true,
  "sources": [
    {
      "document_name": "annual_report.pdf",
      "chunk_index": 12,
      "excerpt": "Q3 total revenue reached $4.2 million..."
    }
  ],
  "agent_trace": [
    "Planner: Analysing question - 'What was the Q3 revenue?'",
    "Retriever: Retrieved 5 chunk(s) from vector store",
    "Reasoning: Synthesised context from 5 chunk(s)",
    "Response: Generated answer (grounded=True)",
    "Verifier: Answer confirmed grounded (verdict=YES)"
  ]
}
```

Response 200 — out-of-scope (refusal):

```
{
  "question": "Who won the 2024 World Cup?",
  "answer": "I could not find this information in the uploaded documents.",
  "is_grounding": false,
  "sources": [],
  "agent_trace": [...]
}
```

Error responses: - **400** — empty question or unsafe prompt detected - **503** — no documents ingested yet, or LLM unavailable

7. Deployment Guide

Local (Development)

```
python main.py # API on :8000
streamlit run ui/streamlit_app.py # UI on :8501
```

Docker Compose (Recommended)

```
cp .env.example .env
docker compose up --build

# Services:
# API → http://localhost:8000
# UI → http://localhost:8501
```

The `docker-compose.yml` defines: - `api` service: FastAPI, port 8000, mounts `./data` volume - `ui` service: Streamlit, port 8501, points `API_BASE_URL` to the api service - Health check: `GET /health` every 30s

Note: Ollama must be running on the host. The containers connect to `host.docker.internal:11434` (or configure `LLM_BASE_URL` accordingly).

Cloud Deployment (Render)

1. Push code to GitHub (already done)
2. Create a new Web Service on render.com pointing to the repo
3. Set build command: `pip install -r requirements.txt`
4. Set start command: `python main.py`
5. Add environment variables in Render dashboard (set `LLM_BASE_URL` to your Groq/OpenAI endpoint)
6. Deploy — Render provides a free tier with HTTPS

Production Checklist

- [] Switch `LLM_BASE_URL` to a cloud provider (Groq or OpenAI) — Ollama is local-only
- [] Set `LLM_API_KEY` to a real API key
- [] Set `CHROMA_PATH` to a persistent volume path
- [] Review `MAX_FILE_SIZE_MB` for your use case

- [] Add authentication if exposing publicly (not included in v1)
- [] Configure log aggregation to capture structured JSON logs

8. Testing Approach

The project has four test layers covering 167 tests total:

Test Layers

Layer	Tests	Speed	Requires
Unit	92	~1s	Nothing — all mocked
Integration	23	~5s	Ollama embed model
E2E	22	~2 min	Full Ollama (LLM + embed)
Regression	23	~2 min	Full Ollama + sample datasets

Running Tests

```
# Fast — no Ollama needed (unit only)
pytest tests/unit/ -m unit

# With Ollama running
pytest tests/integration/ -m integration
pytest tests/e2e/ -m e2e
pytest tests/regression/ -m regression

# Everything
pytest tests/
```

Key Test Scenarios

Grounding invariants (regression): - 5 out-of-scope questions verified to never return fabricated answers - Refusals always have empty `sources[]` - Canonical refusal message is consistent

Format regression matrix: - Each of 6 formats tested: upload → chunk → embed → Q&A - Chunk metadata (source, chunk_index) verified for all formats

Streamlit UI (AppTest): - Health banner: ok / degraded / unreachable scenarios -
Q&A interaction: grounded (success), refusal (info), 503 (error) - Form validation:
empty question shows warning, no API call made

9. Limitations

Technical Limitations

Limitation	Detail
Single user	No concurrent user isolation — all users share one ChromaDB collection. Multiple simultaneous users will see each other's documents.
English only	Models (llama3.2, nomic-embed-text) are optimised for English. Multi-language documents may return poor results.
Local model accuracy	llama3.2 (3B parameters) is a small model. It may fail to extract precise numbers, dates, or specific facts from dense technical documents. Larger models (e.g., llama3.3 70B via Groq) perform significantly better.
PDF tables	Tables in PDFs are extracted as unstructured text. Complex tabular data (merged cells, multi-column layouts) may be misrepresented.
10 MB file limit	Documents larger than 10 MB must be split. This can be raised via <code>MAX_FILE_SIZE_MB</code> but larger documents produce more chunks and slower embedding.
No document deletion	There is no API endpoint to remove specific documents from the vector store. Clearing requires deleting the ChromaDB directory and re-ingesting.
Chunk boundary answers	If an answer spans a chunk boundary, the system may miss it. The 50-token overlap mitigates this but does not eliminate it.

Scope Limitations (v1 by design)

- **No authentication:** the API is open. Production deployments should add API keys or OAuth.
- **No conversation memory:** each question is independent. The system cannot answer follow-up questions like "What about Q4?" without re-stating context.

- **No multi-file queries across collections:** all documents share one ChromaDB collection. There is no way to query only a specific subset of documents.

10. Challenges & How They Were Solved

Challenge 1: Python Logging Reserved Key

Problem: Using `filename` as an `extra` key in Python's `logging.Logger` raised `KeyError: "Attempt to overwrite 'filename' in LogRecord"`. The `filename` field is reserved by Python's `LogRecord` class.

Solution: Renamed all uses of `filename` in log calls to `doc_name`. Also improved the `JsonFormatter` to extract extra fields from `record.__dict__` by excluding standard `LogRecord` fields, rather than checking for a non-existent `record.extra` attribute.

Lesson: Always check Python's logging module reserved field names (`filename`, `lineno`, `funcName`, `pathname`, `module`, `name`, `levelname`, `message`, `asctime`) before using them as extra keys.

Challenge 2: LangChain Deprecation Cascade

Problem: LangChain releases frequently deprecate older import paths. During implementation, several imports stopped working: - `langchain.schema.Document` → moved to `langchain_core.documents` - `langchain.schema.HumanMessage` → moved to `langchain_core.messages` - `langchain_community.vectorstores.Chroma` → moved to `langchain_chroma` (separate package) - `langchain_community.chat_models.ChatOpenAI` → moved to `langchain_openai` (separate package)

Solution: Audited all imports and updated to current `langchain_core` and provider-specific packages (`langchain_chroma`, `langchain_openai`). Added these to `requirements.txt`.

Lesson: When using LangChain, prefer `langchain_core.*` imports over `langchain.*` and install provider-specific packages separately (`langchain-openai`, `langchain-chroma`) rather than relying on `langchain-community`.

Challenge 3: Ollama Homebrew Installation Missing Binaries

Problem: Ollama installed via `brew install ollama` (formula version 0.30.6) was missing the `llama-server` binary required to run models. The CLI tool installed but model inference failed with: `error starting llama-server: llama-server binary not found`.

Solution: Installed the full Ollama desktop application via `brew install --cask ollama`, which includes all required binaries. The formula and cask are separate packages.

Lesson: Ollama's Homebrew formula installs only the CLI client. For actual model inference on macOS, the cask (Ollama.app) is required.

Challenge 4: Large Document Embedding Timeout

Problem: The Nobel Prize dataset (`nobel_prizes.json`, 528 KB) produced 5,064 chunks. Attempting to embed all 5,064 chunks in a single ChromaDB `add_documents()` call caused Ollama to return an EOF error on the tokenize endpoint after ~25 seconds.

Solution: Implemented batched embedding in `VectorStore.add_chunks()` — processes 100 chunks per Ollama call, then stores them all in ChromaDB. This keeps each individual embedding call well within Ollama's timeout.

Lesson: Local embedding models have practical limits on batch size. For production systems with large documents, embedding should always be batched, regardless of whether the API nominally supports large batches.

Challenge 5: ChromaDB TypedDict Incompatibility

Problem: Tests for agent nodes used a helper `_state(**kwargs)` that tried to spread extra keys into a `TypedDict`. Python's `TypedDict` does not support `**kwargs` spreading — unknown keys are silently ignored at runtime but cause test logic errors.

Solution: Rewrote the test helper to construct the full `AgentState` dict explicitly, then apply overrides with `{**base_state, **overrides}`.

Lesson: `TypedDict` in Python is a static type hint, not a runtime class. It does not enforce keys at runtime but will not accept arbitrary `**kwargs` in the way a dataclass or Pydantic model would.

Challenge 6: Streamlit AppTest Mocking Scope

Problem: When testing the Streamlit UI with `AppTest`, mocking higher-level functions (e.g., `patch("ui.streamlit_app.ask_question", ...)`) did not work for interactions after `.click().run()`. `AppTest` re-executes the script from scratch on every `.run()` call, loading fresh module state that bypasses function-level mocks.

Solution: Patched `requests.get` and `requests.post` directly — the lowest-level HTTP calls that the UI makes. These patches persist across `.run()` calls because they're at the `requests` library level, not the module level.

Lesson: Streamlit's `AppTest` creates a fresh Python execution context on each `.run()`. Always mock at the library (I/O) level rather than the application function level when testing with `AppTest`.

Challenge 7: Verifier Clears Sources on Refusal

Problem: The Retriever always populates `sources` with the top-K nearest chunks (by cosine similarity) even when the question is out-of-scope. When the Response agent returned a refusal, the `sources` field still contained unrelated chunks — misleading the client into thinking those chunks were cited evidence.

Solution: Modified `response_node` to clear `sources = []` when `is_grounded=False`. Similarly, `verifier_node` clears sources if the verification verdict is NO.

Lesson: In a multi-agent pipeline, each agent's output must be consistent. The Retriever's job is to find similar chunks; the Response agent's job is to determine whether those chunks actually answer the question. When they don't, downstream data (sources) must be reset to reflect reality.

11. Assumptions

1. **English documents:** the system assumes documents are primarily in English. The embedding model and LLM are English-optimised.
2. **Honest documents:** the system does not validate whether document content is accurate. If a document contains incorrect information, the system will faithfully retrieve and cite it.
3. **Single-user operation:** no concurrent user isolation was designed. The system is intended for a single user or small team using it sequentially.

4. **Ollama on same machine:** in development, Ollama is expected to run on `localhost:11434`. The `LLM_BASE_URL` env var makes this configurable for other setups.
 5. **Adequate hardware:** `llama3.2` (3B parameters) requires at least 4 GB RAM available. `nomic-embed-text` requires ~500 MB. Total: ~6–8 GB recommended.
 6. **Document text extractability:** PDF files must contain extractable text. Scanned PDFs (image-only) cannot be processed without OCR (not included in v1).
 7. **Chunk size sufficient for answers:** the 500-token chunk size with 50-token overlap is designed for typical enterprise document paragraphs. Very long, continuous answers may be split across chunks.
-

12. Future Improvements

Short-Term (v1.1)

- **OCR support:** integrate `pytesseract` or `easyocr` to extract text from scanned PDF images
- **Document management:** `DELETE /document/{id}` endpoint to remove specific documents from the vector store
- **Conversation memory:** maintain conversation history so follow-up questions work naturally ("What about Q4?" without restating context)
- **Relevance threshold:** configurable minimum similarity score — chunks below the threshold are excluded even if they are the "top-K"

Medium-Term (v2.0)

- **User authentication:** API key or OAuth2 for multi-user deployments
- **Per-user collections:** separate ChromaDB collections per user so documents don't mix
- **Streaming responses:** stream LLM output token-by-token instead of waiting for the full answer
- **Table-aware parsing:** use `camelot` or `pdfplumber` for structured table extraction from PDFs
- **Feedback loop:** thumbs up/down on answers to identify where the system fails

Long-Term

- **Multi-modal documents:** support images, charts, and diagrams via vision models

- **Agent planning:** genuine multi-step planning for complex questions requiring synthesis across many documents
 - **Hybrid retrieval:** combine dense vector search with BM25 keyword search (hybrid RAG) for better precision on technical terms
 - **Fine-tuning:** fine-tune the embedding model on domain-specific terminology for specialised enterprise use cases
-